



Academy of  
Engineering

(An Autonomous Institute Affiliated to Savitribai Phule Pune University)

# **AIML Practical 2**

## **Prolog activity**

**Name: Harshini Ramdas Bhandary**

**PRN: 202401040151**

**Roll no: 35**

**Division: A    Batch: A2**

# 1. Title

## Student Mentor Program Using Prolog

# 2. Aim

To develop a simple Student Mentor Program using Prolog that identifies suitable mentors for students based on the subjects they know and the subjects other students need. The program also checks common free days, study-together relationships, learning modes, and overall mentor compatibility.

# 3. Experiment Objectives

The main objectives of this experiment are:

1. To understand how facts and rules are represented in Prolog.
2. To store information about students, subjects, availability, and learning modes.
3. To find a student who can mentor another student in a required subject.
4. To find a common day on which a mentor and student are both available.
5. To identify students who can mentor each other.
6. To find students having the same learning mode.
7. To combine subject knowledge, availability, and learning mode to find a good mentor match.
8. To understand how Prolog uses logical rules and queries to obtain results.

# 4. Experiment Outcomes

After completing this experiment, the following outcomes can be achieved:

- Student information can be represented using Prolog facts.
- A suitable mentor can be identified automatically from the given knowledge.
- Common available days can be found for mentoring sessions.
- Students who can study together can be identified.
- Students using the same mode, such as online or offline, can be matched.
- A better mentor-student combination can be obtained by checking multiple conditions together.
- The experiment demonstrates how Prolog can solve relationship-based problems using logical reasoning.

# 5. Conceptual Details

## 5.1 Prolog

Prolog is a logic programming language. Instead of writing a sequence of instructions like in conventional programming, we describe **facts and rules**. Prolog then uses these facts and rules to answer questions called **queries**.

In this experiment, information about students is stored as facts.

For example:

```
student(harshini, 3).
```

This represents that Harshini is a student of the third year.

Similarly:

```
knows(harshini, maths).
```

means that Harshini knows Maths.

The program also stores the subjects that students need, their free days, and their preferred learning mode.

## 5.2 Facts

Facts are statements that are directly given to the Prolog system.

The program contains facts for:

- Students
- Subjects known by students
- Subjects needed by students
- Available days
- Learning modes

For example, the program stores that Harshini knows Maths and Java and needs Networks.

Availability is represented using `free_on`, such as:

```
free_on(harshini, monday).
```

The learning mode is represented using `mode`, such as:

```
mode(harshini, online).
```

## 5.3 Rules

Rules are used to derive new information from the existing facts.

The first important rule is mentor.

mentor(X, Y) :-

knows(X, Subject),

needs(Y, Subject),

$X \neq Y$ .

This rule checks three conditions:

1. X knows a subject.
2. Y needs the same subject.
3. X and Y must be different students.

If all three conditions are satisfied, X can mentor Y.

## 5.4 Session Matching

The session rule checks whether the mentor and student are both free on the same day.

session(X, Y, Day) :-

mentor(X, Y),

free\_on(X, Day),

free\_on(Y, Day).

Therefore, a session is possible only when:

- X can mentor Y.
- X is free on the selected day.
- Y is also free on that day.

## 5.5 Study Together

The program also checks whether two students can mentor each other.

study\_together(X, Y) :-

mentor(X, Y),

mentor(Y, X),

$X \neq Y$ .

Here, both directions are checked. X must be able to mentor Y, and Y must also be able to mentor X.

## 5.6 Same Learning Mode

The `same_mode` rule compares the learning mode of two students.

`same_mode(X, Y) :-`

`mode(X, M),`

`mode(Y, M),`

`X \= Y.`

If both students have the same mode and they are different people, the rule considers them to have the same learning mode.

## 5.7 Good Match

The `good_match` rule combines different conditions.

`good_match(X, Y, Day) :-`

`mentor(X, Y),`

`free_on(X, Day),`

`free_on(Y, Day),`

`same_mode(X, Y).`

A good match therefore requires:

- X can mentor Y.
- Both are free on the same day.
- Both have the same learning mode.

This makes the matching more useful than checking subject knowledge alone.

# 6. Algorithm / Flowchart Details

## Algorithm

**Step 1:** Start the Prolog program.

**Step 2:** Store the student details as facts.

**Step 3:** Store the subjects known by each student.

**Step 4:** Store the subjects required by each student.

**Step 5:** Store the available days of the students.

**Step 6:** Store the learning mode of each student.

**Step 7:** Apply the mentor rule.

**Step 8:** Compare the subjects known by one student with the subjects needed by another student.

**Step 9:** If the subject matches and the students are different, identify the first student as a possible mentor.

**Step 10:** Apply the session rule to check whether both students are available on a common day.

**Step 11:** Apply the study\_together rule to find students who can mentor each other.

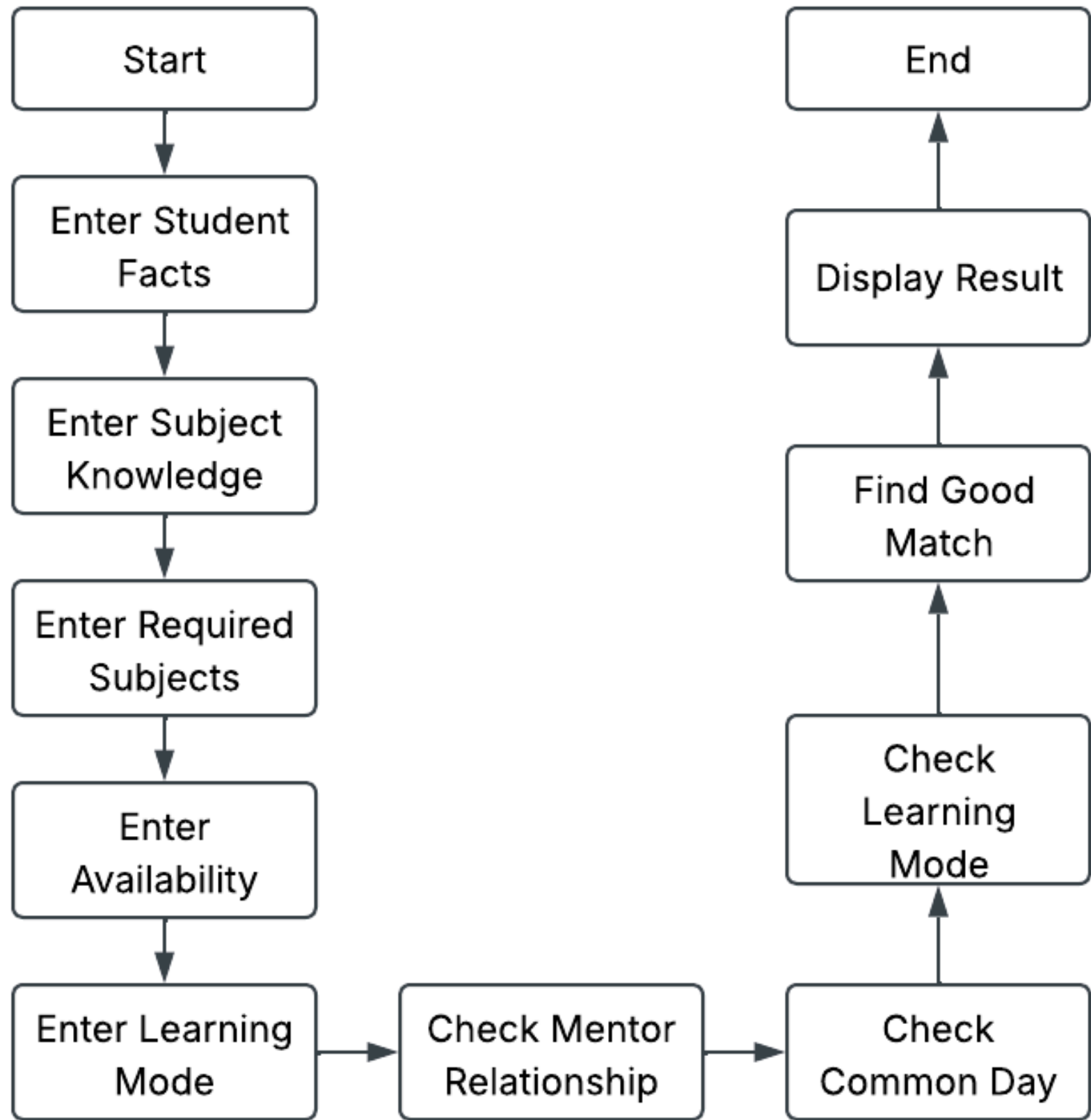
**Step 12:** Apply the same\_mode rule to find students using the same learning mode.

**Step 13:** Apply the good\_match rule to combine mentoring ability, common availability, and learning mode.

**Step 14:** Display the results obtained from the required query.

**Step 15:** Stop.

## Flowchart



# 7. Code (Implementation)

## CODE:

```
SWISH File Edit Examples Help
Harshini Ramdas Bhandary +
1 % Student Mentor Program
2
3 % 1. Relations
4 student(harshini, 3).
5 student(riya, 2).
6 student(parth, 3).
7 student(sushrut, 1).
8 student(omkar, 2).
9
10 knows(harshini, maths).
11 knows(harshini, java).
12 knows(riya, maths).
13 knows(riya, dbms).
14 knows(parth, java).
15 knows(parth, networks).
16 knows(sushrut, dbms).
17 knows(omkar, networks).
18 knows(omkar, maths).
19
20 needs(harshini, networks).
21 needs(riya, java).
22 needs(parth, dbms).
23 needs(sushrut, maths).
24 needs(omkar, java).
25
26 free_on(harshini, monday).
27 free_on(harshini, friday).
28 free_on(riya, wednesday).
29 free_on(parth, monday).
30 free_on(parth, saturday).
31 free_on(sushrut, friday).
32 free_on(omkar, wednesday).
33
34 mode(harshini, online).
35 mode(riya, offline).
36 mode(parth, online).
37 mode(sushrut, offline).
38 mode(omkar, online).
39
40
```

```
SWISH File Edit Examples Help
Harshini Ramdas Bhandary +
39
40
41 % 2. Define the relations
42
43 mentor(X, Y) :-
44     knows(X, Subject),
45     needs(Y, Subject),
46     X \= Y.
47
48 session(X, Y, Day) :-
49     mentor(X, Y),
50     free_on(X, Day),
51     free_on(Y, Day).
52
53 study_together(X, Y) :-
54     mentor(X, Y),
55     mentor(Y, X),
56     X \= Y.
57
58 same_mode(X, Y) :-
59     mode(X, M),
60     mode(Y, M),
61     X \= Y.
62
63 good_match(X, Y, Day) :-
64     mentor(X, Y),
65     free_on(X, Day),
66     free_on(Y, Day),
67     same_mode(X, Y).
68
```

## OUTPUT:

```
mentor(X, sushrut).
X = harshini
X = riya
X = omkar

?- mentor(X, sushrut).
```

```
mentor(X, riya).
X = harshini
X = parth
false

?- mentor(X, riya).
```

 `session(X, sushrut, Day).`

**Day** = friday,  
**X** = harshini  
**false**

?- `session(X, sushrut, Day).`

 `study_together(X, Y).`

**X** = harshini,  
**Y** = omkar  
**X** = riya,  
**Y** = parth  
**X** = parth,  
**Y** = riya  
**X** = omkar,  
**Y** = harshini

**false**

?- `study_together(X, Y).`

 `good_match(X, Y, Day).`

**Day** = monday,  
**X** = parth,  
**Y** = harshini  
**false**

?- `good_match(X, Y, Day).`

 `same_mode(X, Y).`

**X** = harshini,  
**Y** = parth  
**X** = harshini,  
**Y** = omkar  
**X** = riya,  
**Y** = sushrut  
**X** = parth,  
**Y** = harshini  
**X** = parth,  
**Y** = omkar  
**X** = sushrut,  
**Y** = riya  
**X** = omkar,  
**Y** = harshini  
**X** = omkar,  
**Y** = parth

**false**

?- `same_mode(X, Y).`

## 8. Result Analysis

The program successfully represents a student mentoring system using Prolog facts and logical rules.

For example, the query:

```
?- mentor(X, sushrut).
```

checks which student knows the subject required by Sushrut. Since Sushrut needs Maths, Prolog searches the knows facts and finds students who know Maths.

The query:

```
?- mentor(X, riya).
```

checks who can help Riya with Java.

The session query goes one step further by checking whether the mentor and student are available on the same day.

The study\_together query checks for a two-way mentoring relationship. This means both students must have a subject that the other student needs.

The same\_mode query compares whether two different students use the same learning mode.

Finally, good\_match combines these conditions. It gives a match only when the mentor relationship, common availability, and learning mode conditions are satisfied.

Thus, the program does not simply store student information. It uses the stored information to derive useful relationships automatically through Prolog rules.

## 9. Conclusion

The Student Mentor Program was successfully implemented using Prolog. The experiment demonstrated how facts can be used to store student information and how rules can be used to derive new relationships from those facts.

The program can identify possible mentors based on subject knowledge, check common available days, find two-way study relationships, compare learning modes, and identify better mentor-student combinations.

This experiment helped in understanding the basic working of logic programming, especially the use of **facts, rules, variables, conditions, and queries** in Prolog.

Overall, Prolog provides a simple way to represent this type of relationship-based problem because the programmer mainly describes the available knowledge and the conditions for a valid match, while Prolog performs the logical search.